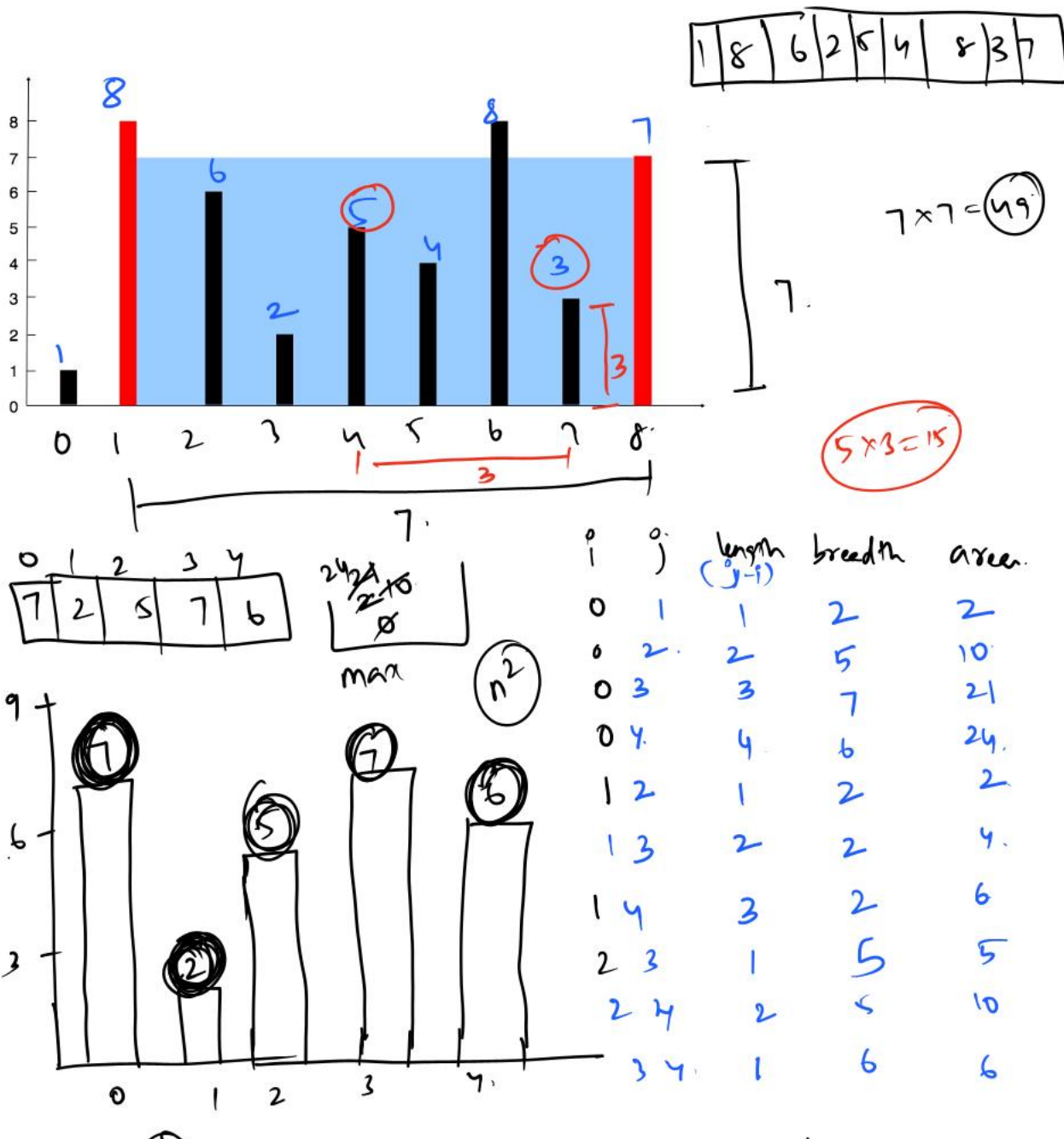


Solved

 Hint

Notice that you may not slant the container.

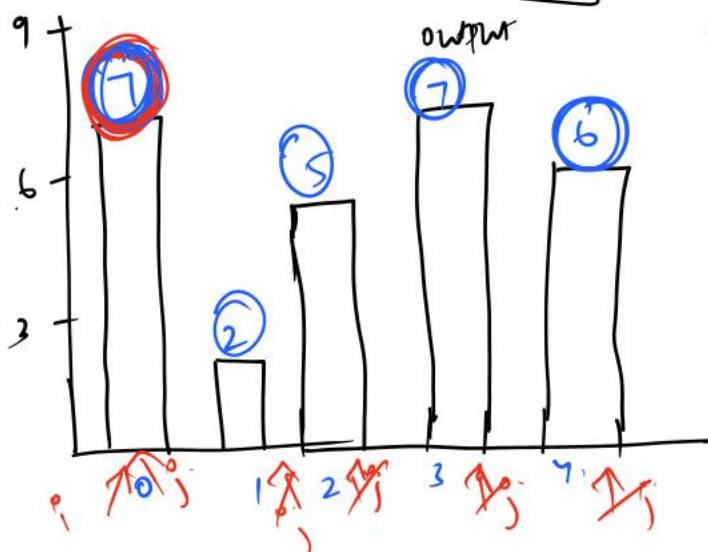


$n \rightarrow (10^5)$
 $T.C \rightarrow n^2 \rightarrow (10^5)^2 \rightarrow 10^{10} > 10^8$
 TILE

$13ec \rightarrow 10^9$
 $25^2 \rightarrow 10^8$

0 1 2 3 4
 7 2 5 7 6

24



i	j	length	breadth	area
0	4	4	6	24
0	3	3	7	21
0	2	2	5	10
0	1	1	2	2

$i == j \rightarrow$ stop calculating area.

$i \neq j \rightarrow$ calculate area

class Solution:

```

def maxArea(self, height: List[int]) -> int:
    output = 0
    i = 0
    j = len(height) - 1
    while i != j:
        length = j - i
        breadth = min(height[i], height[j])
        area = length * breadth
        output = max(area, output)
        if height[i] < height[j]:
            i += 1
        else:
            j -= 1
    return output
    
```

$\rightarrow p \cdot n \cdot 11$

$\rightarrow T.C \rightarrow O(n)$

$\rightarrow S.C \rightarrow O(1)$

TwoPointer

283. Move Zeroes

Solved

Easy

Topics

Companies

Hint

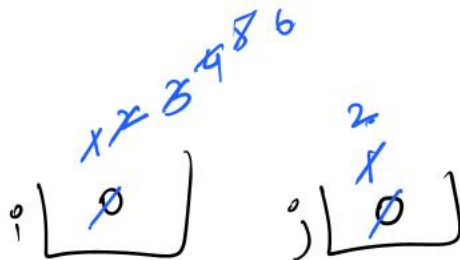
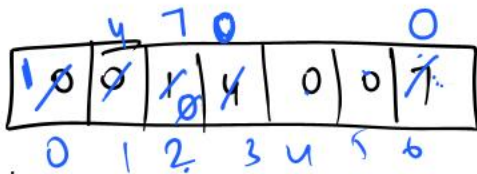
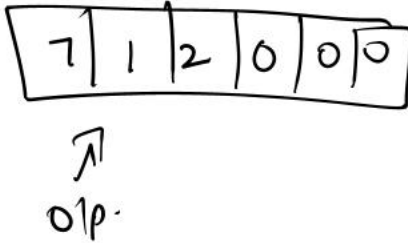
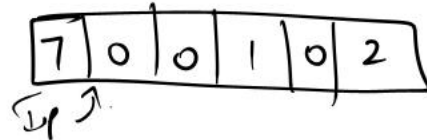
Given an integer array `nums`, move all `0`'s to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

Input: `nums = [0,1,0,3,12]`

Output: `[1,3,12,0,0]`



initialize `j` $\rightarrow$ 0
 # Iterate through list.
 # if `i`th element $\rightarrow$ non zero.
 # swap (`i`, `j`)
 # increment `j` by 1

Move zeroes to end.

Move non zeros $\rightarrow$ front

1st non zero $\rightarrow$ 0 index.
 2nd non zero $\rightarrow$ 1 index.
 3rd non zero $\rightarrow$ 2 index.

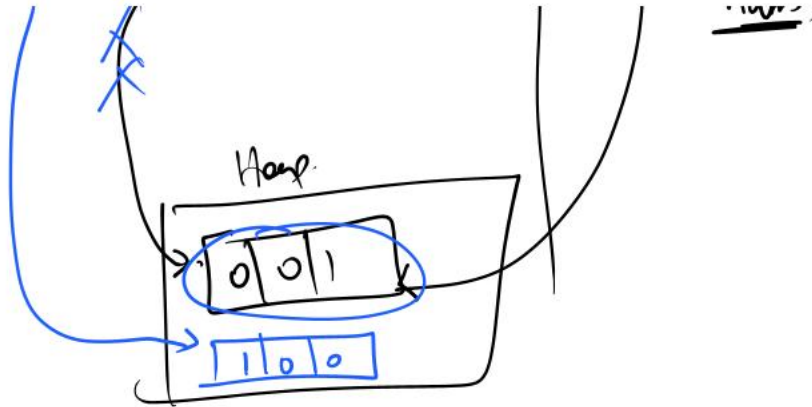
$0 < \text{nonZero} < n$
 nonZero

def moveZeroes(nums) $\rightarrow$ None:

`nums1 = [1,0,0]`
`nums = nums1`

class Prob283:

`s = Solution()`
`nums = [0,1,0,2]`
`s.moveZeroes(nums)`



```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        j = 0
        for i in range(len(nums)):
            if nums[i] != 0:
                nums[i], nums[j] = nums[j], nums[i]
                j += 1
```

TWO POINTER

p.No: 283

T.C $\rightarrow O(n)$

S.C $\rightarrow O(1)$

905. Sort Array By Parity

Solved

Easy

Topics

Companies

Given an integer array `nums`, move all the even integers at the beginning of the array followed by all the odd integers.

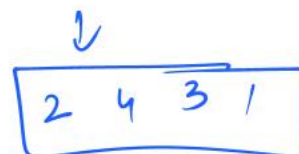
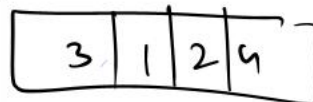
Return **any array** that satisfies this condition.

Example 1:

Input: `nums = [3,1,2,4]`

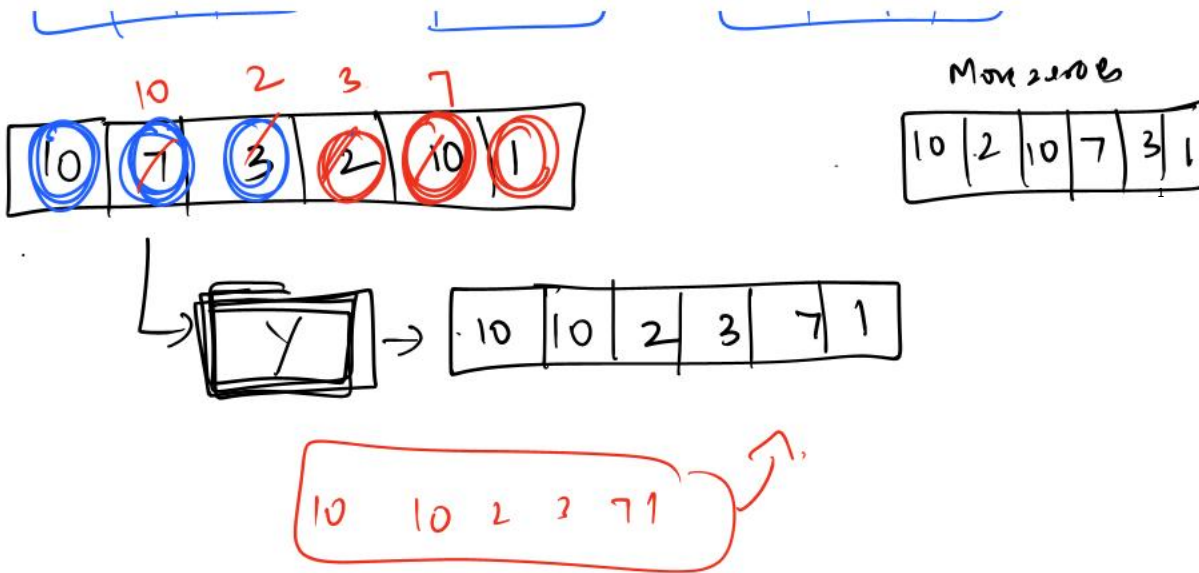
Output: `[2,4,3,1]`

Explanation: The outputs `[4,2,3,1]`, `[2,4,1,3]`, and `[4,2,1,3]` would also be accepted.



Move 2 and 4
approach





```
class Solution:
    def sortArrayByParity(self, nums: List[int]) -> List[int]:
        j = len(nums) - 1
        i = 0
        while i < j:
            if nums[i] % 2 == 0:
                i += 1
            elif nums[j] % 2 == 1:
                j -= 1
            else:
                nums[i], nums[j] = nums[j], nums[i]
                i += 1
                j -= 1
        return nums
```

→ P.No: 905

→ T.C → $O(n)$

→ S.C → $O(1)$

941. Valid Mountain Array

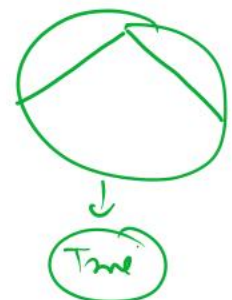
Solved

Easy Topics Companies Hint

Given an array of integers `arr`, return `true` if and only if it is a valid mountain array.

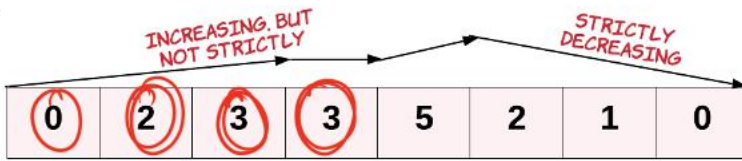
Recall that `arr` is a mountain array if and only if:

- `arr.length >= 3`
- There exists some `i` with `0 < i < arr.length - 1` such that:
 - `arr[0] < arr[1] < ... < arr[i - 1] < arr[i]`
 - `arr[i] > arr[i + 1] > ... > arr[arr.length - 1]`

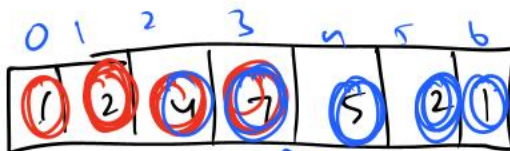
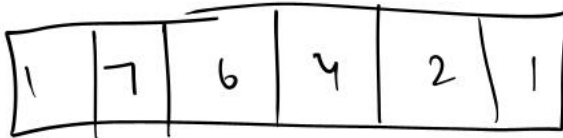
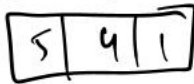




MOUNTAIN ARRAY

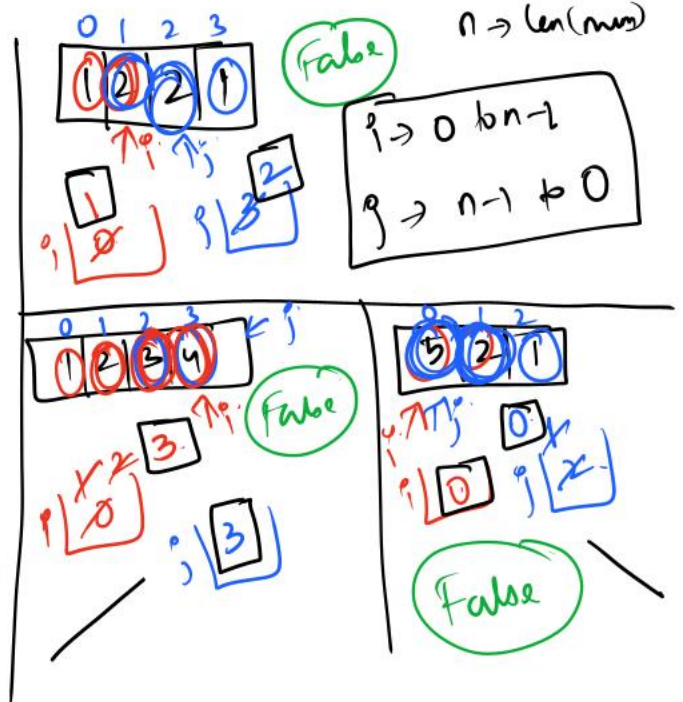


NOT A MOUNTAIN ARRAY



True
strictly increasing

3
strictly decreasing



```
class Solution:
    def validMountainArray(self, arr: List[int]) -> bool:
        n = len(arr)
        # Strictly increasing -> 0 to n - 1
        i = 0
        while i < n - 1 and arr[i] < arr[i + 1]:
            i += 1
        if i == 0 or i == n - 1:
            return False
        # Strictly decreasing -> i to n - 1
        while i < n - 1 and arr[i] > arr[i + 1]:
            i += 1
        return i == n - 1
```

```
class Solution:
    def validMountainArray(self, arr: List[int]) -> bool:
        n = len(arr)
        # Strictly increasing -> 0 to n - 1
        i = 0
        while i < n - 1 and arr[i] < arr[i + 1]:
            i += 1
        # Strictly decreasing -> n - 1 to 0
        j = n - 1
        while j > 0 and arr[j] < arr[j - 1]:
            j -= 1
        if i != j or i == 0 or j == n - 1:
            return False
        else:
            return True
```

$K + (N) \times K$

$K + (N) \times K$

P.No: (941) T.C $\rightarrow O(N)$. S.C $\rightarrow O(1)$.

3637. Trionic Array I

Solved

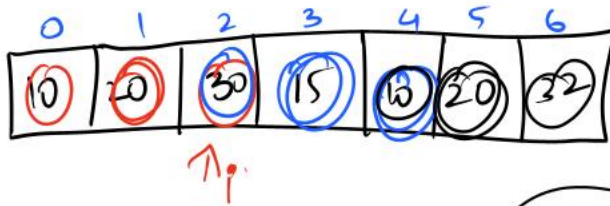
Easy Topics Companies Hint

You are given an integer array `nums` of length `n`.

An array is **trionic** if there exist indices $0 < p < q < n - 1$ such that:

- `nums[0...p]` is **strictly increasing**,
- `nums[p...q]` is **strictly decreasing**,
- `nums[q...n - 1]` is **strictly increasing**.

Return `true` if `nums` is trionic, otherwise return `false`.



$i \times 2$ 3 4 5 6

$i \times 2$

$i = n - 1$

```
class Solution:
    def isTrionic(self, nums: List[int]) -> bool:
        n = len(nums)
        # Strictly increasing
        i = 0
        while i < n - 1 and nums[i] < nums[i + 1] :
            i += 1
        if i == 0 or i == n - 1 :
            return False
        # Strictly decreasing
        while i < n - 1 and nums[i] > nums[i + 1] :
            i += 1
        if i == n - 1 :
            return False
        # Strictly increasing
```

$\rightarrow$ P.No: (3637)

$\rightarrow$ T.C $\rightarrow O(N)$

$\rightarrow$ S.C $\rightarrow O(1)$

$K + (N) \times K$


```

n-p-1 while i < n - 1 and nums[i] < nums[i + 1] :
        i += 1
    return i == n - 1

```

RECURSION

↳ PMTE → Principle of Mathematical Induction.

$$\sum n = \frac{n(n+1)}{2}$$

③ → $\sum 1 = 1$

→ Assume $\sum k = \frac{k(k+1)}{2}$

→ Prove for $k+1$

$$\sum_{k+1} = \frac{(k+1)(k+2)}{2}$$

$$\sum_{k+1} = \sum k + k+1$$

$$= \frac{k(k+1)}{2} + k+1$$

$$= \frac{k(k+1) + 2(k+1)}{2}$$

$$= \frac{(k+1)(k+2)}{2}$$

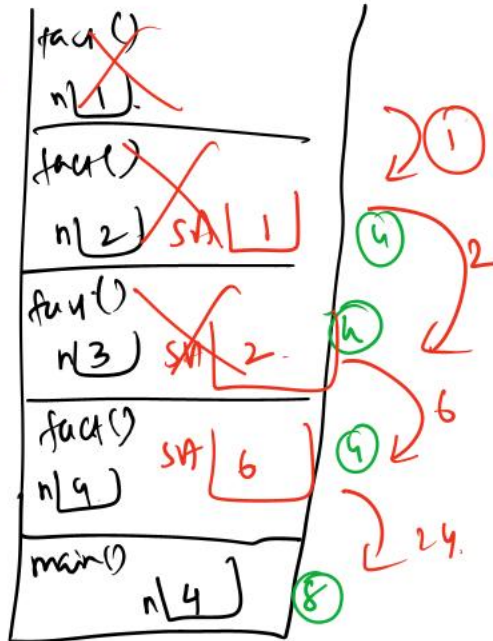
```

1 def fact(n) :
2     if n == 1 :
3         return n
4     smallAns = fact(n - 1)
5     return n * smallAns
6
7 n = 4
8 print(fact(n))

```

→ Base Case
Inductive Hypothesis
Induction step

Ans: 24



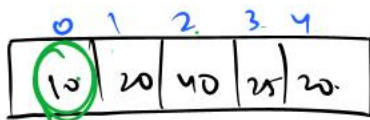

```

2  def printN(n) :
3      if n == 0 :
4          return
5      printN(n - 1)
6      print(n , end = " ")
7
8  n = 4
9  printN(n)

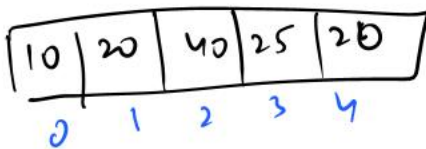
```

1 2 3 4

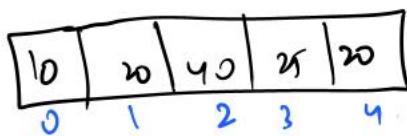
Complete



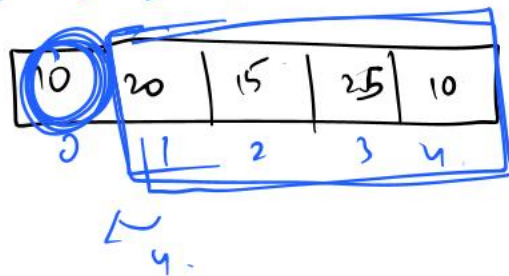
elem | 25 | o/p: 3



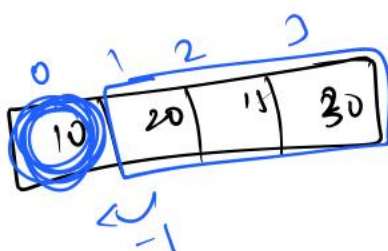
elem | 20 | o/p: 1



elem | 30 | o/p: -1



elem | 10 | o/p: 0

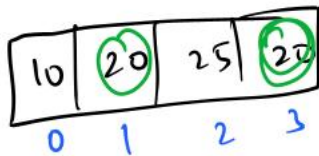


elem | 10 | i | 0

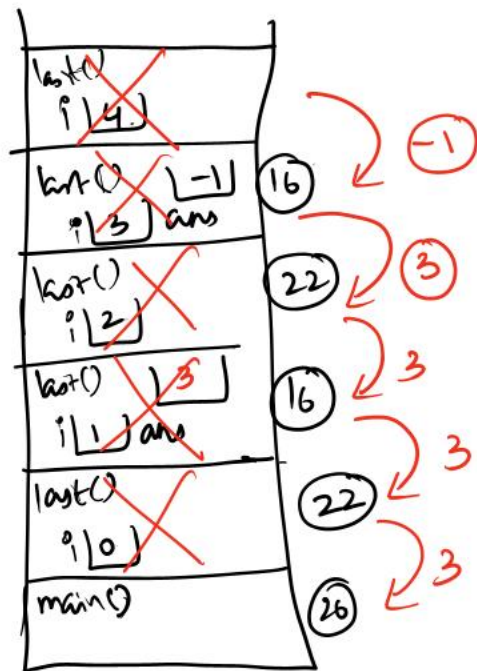
```

12 def lastOccurrence(nums, element, i):
13     if i == len(nums):
14         return -1
15     if nums[i] == element:
16         ans = lastOccurrence(nums, element, i + 1)
17         if ans == -1:
18             return i
19         else:
20             return ans
21     else:
22         return lastOccurrence(nums, element, i + 1)
23
24 nums = [10, 20, 25, 20]
25 element = 20
26 print(lastOccurrence(nums, element, 0))

```



element 20
 3



70. Climbing Stairs

Solved ✓

Easy

Topics

Companies

Hint

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

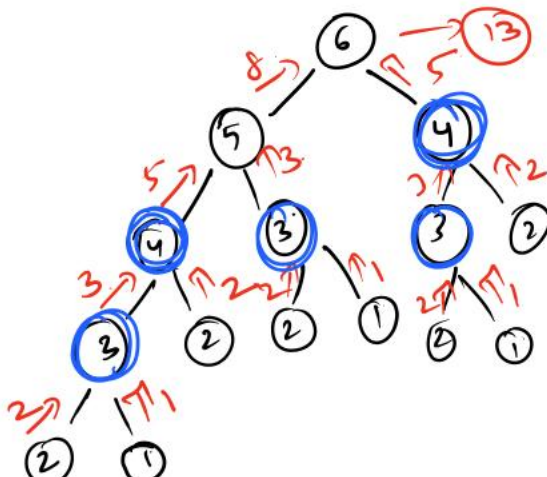
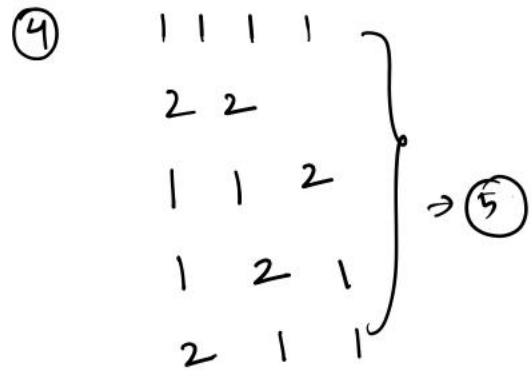
Example 1:

Input: $n = 2$

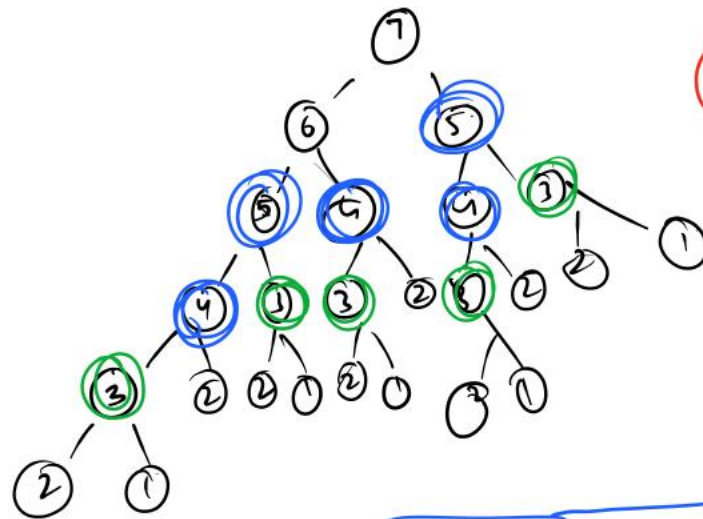
Output: 2

Explanation: There are two ways to climb to the top.

1. 1 step + 1 step
2. 2 steps



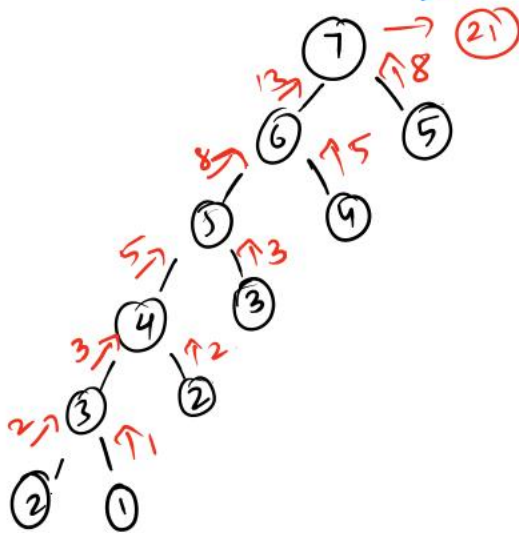
① → ①
 ② → ① + ②



DP → Dynamic Programming

↓
Repeated sub problems

↓
Storing the Repeated sub problems in a DS



3:3
4:5
5:8
6:13

```
class Solution:
    def climbStairs(self, n: int) -> int:
        def helper(n, dp):
            if n == 1 or n == 2:
                return n
            ans1, ans2 = 0, 0
            if n - 1 in dp:
                ans1 = dp[n - 1]
            else:
                ans1 = helper(n - 1, dp)
                dp[n - 1] = ans1 # Storing n - 1's ans in dictionary
            if n - 2 in dp:
                ans2 = dp[n - 2]
            else:
                ans2 = helper(n - 2, dp)
                dp[n - 2] = ans2 # Storing n - 2's ans in dictionary
            return ans1 + ans2
        dp = {}
        return helper(n, dp)
```

RECURSIVE DP

→ P.No: 70

→ T.C → $O(n)$

→ S.C → $O(n)$

↓
Call stack space + Dictionary size

```
class Solution:
```

```
def climbStairs(self, n: int) -> int:
```

```
    dp = [1, 2]
```

```
    for i in range(2, n + 1):
```

```
        dp.append(dp[i - 1] + dp[i - 2])
```

```
    return dp[n - 1]
```

ITERATIVE DP

→ P.W: 70

→ T.C $\rightarrow O(n)$

↓
S.C $\rightarrow O(n)$. → Array's size